Chapter 03

Dependency management, Exception Handling, Logging

The examples are publicly available on:
https://github.com/skanderturki/se411

Introduction to Dependency Management

- Managing dependencies is a difficult task that can be time consuming if done manually:
  - How to make sure all the team members are using the same version of some library?
  - How to allow a developer to setup a new development environment without having to search/download/install/configure all the dependencies of the project?
  - How to easily manage the migration to new versions of dependencies at the level of the project configuration of all team members without time loss?

Introduction to Dependency Management (2)

- Dependency management tools allow to automate these tasks by providing a declarative way of specifying the project dependencies.

- Maven and Gradle are the most popular tools to manage dependencies in Java projects.

Importing a dependency or a plugin using Maven

The pom.xml contains all the dependencies to (libraries and plugins).

Plugins contain scripts that are executed by Maven

Plugins define **goals**: the **codehaus mojo exec** plugin defines the **java** goal that allows to execute java programs:

command to execute: > mvn **exec:java**

```
<plugins>
   <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.6.3</version>
      <configuration>
         <mainClass>edu.psu.se411.App</mainClass>
      </configuration>
   </plugin>
</plugins>
```

Here, we are configuring our project to use version 3.6.3 of the “codehaus majo exec” plugin, the plugin will be automatically downloaded by Maven.

Importing a dependency or a plugin using Maven

In this example, we are importing different libraries into our Spring project.

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jdbc</artifactId>
    </dependency>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version> <!-- Use latest version -->
    </dependency>
</dependencies>
```

Maven build lifecycle



It constitutes a series of stages, also called phases. that get executed in the same order. Every Maven project has the following three built-in lifecycles:

- ***default***: This **lifecycle** handles the compiling, packaging, and deployment of a Maven project.
- ***clean***: This **lifecycle** handles the deletion of temporary files and generated artifacts from the target directory.
- ***site***: This **lifecycle** handles the generation of documentation and site generation.

These are Maven **phases** of the default lifecycle (a phase runs all previous phases):

- ***validate***: Runs checks to ensure that the project is correct and that all dependencies are downloaded and available.
- ***compile***: Compiles the source code.
- ***test***: Runs unit tests using frameworks. This step doesn't require that the application be packaged.
- ***package***: Assembles compiled code into a distributable format, such as JAR or WAR.
- ***install***: Installs the packaged archive into a local Maven repository (~/.m2/repository). The archive is now available for use by any project running on that machine.
- **c** other teams

Example of core plugins: site plugin

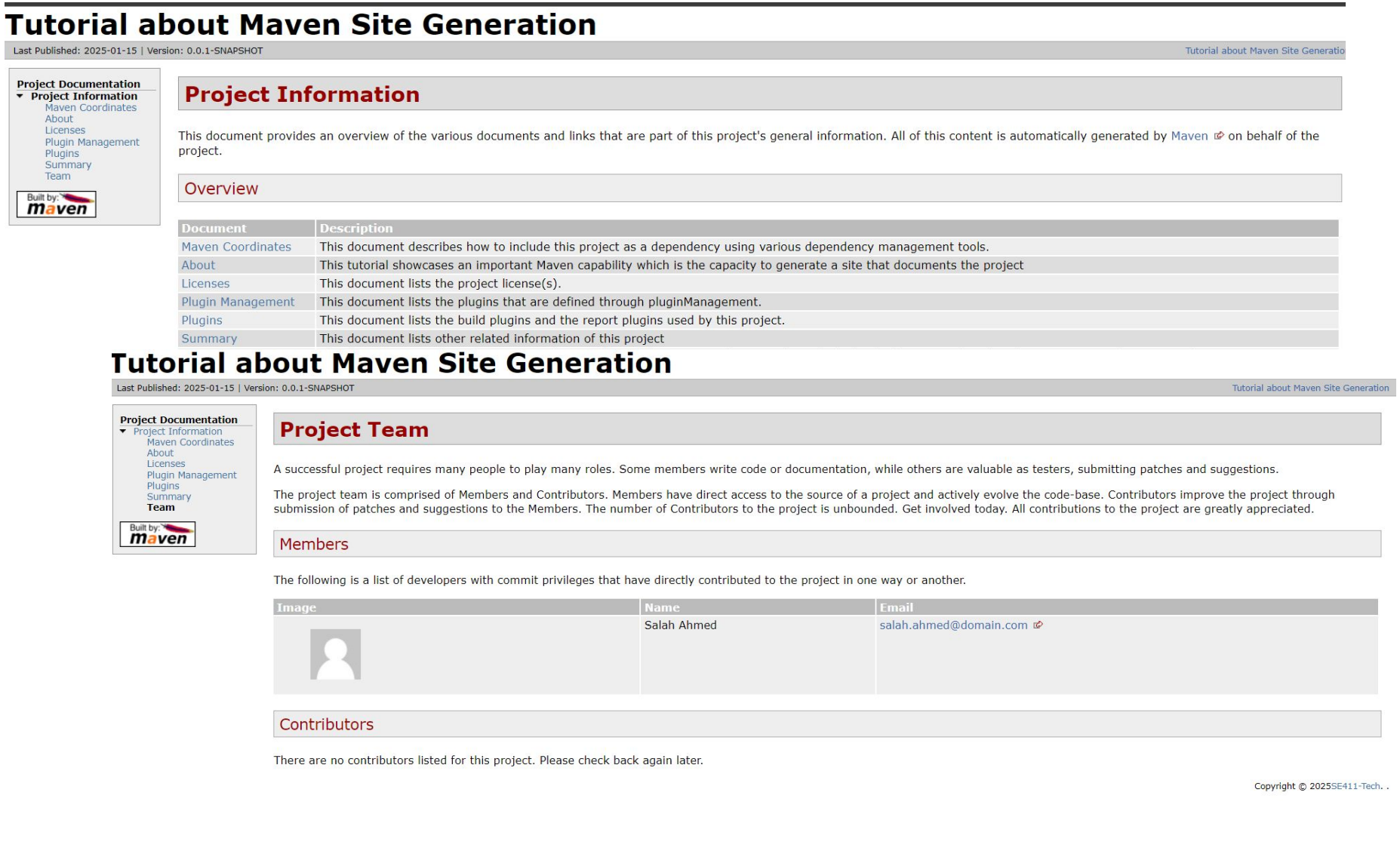
```
<name>Tutorial about Maven Site Generation</name>
<description>This tutorial showcases an important Maven capability which is the capacity to generate a site that documents the project</description>
<url>https://sites.google.com/view/se411/home</url>
<licenses>
    <license>
      <name>MIT License</name>
      <comments>This is an open-source project</comments>
    </license>
</licenses>
<organization>
    <name>SE411-Tech</name>
    <url>https://github.com/skanderturki/se411</url>
</organization>
<developers>
    <developer>
        <name>Salah Ahmed</name>
        <email>salah.ahmed@domain.com</email>
    </developer>
</developers>
```

In pom.xml, we can specify details about the project

Source Code: maven-02-site

Example of plugins: core site plugin

- After running the site goal:



Tutorial about Maven Site Generation

Last Published: 2025-01-15 | Version: 0.0.1-SNAPSHOT — Tutorial about Maven Site Generatio

Project Documentation
- **Project Information**
  - Maven Coordinates
  - About
  - Licenses
  - Plugin Management
  - Plugins
  - Summary
  - Team

Built by: maven

Project Information

This document provides an overview of the various documents and links that are part of this project's general information. All of this content is automatically generated by Maven on behalf of the project.

Overview

Document	Description
Maven Coordinates	This document describes how to include this project as a dependency using various dependency management tools.
About	This tutorial showcases an important Maven capability which is the capacity to generate a site that documents the project
Licenses	This document lists the project license(s).
Plugin Management	This document lists the plugins that are defined through pluginManagement.
Plugins	This document lists the build plugins and the report plugins used by this project.
Summary	This document lists other related information of this project

Tutorial about Maven Site Generation

Last Published: 2025-01-15 | Version: 0.0.1-SNAPSHOT — Tutorial about Maven Site Generation

Project Documentation
- Project Information
  - Maven Coordinates
  - About
  - Licenses
  - Plugin Management
  - Plugins
  - Summary
  - **Team**

Built by: maven

Project Team

A successful project requires many people to play many roles. Some members write code or documentation, while others are valuable as testers, submitting patches and suggestions.

The project team is comprised of Members and Contributors. Members have direct access to the source of a project and actively evolve the code-base. Contributors improve the project through submission of patches and suggestions to the Members. The number of Contributors to the project is unbounded. Get involved today. All contributions to the project are greatly appreciated.

Members

The following is a list of developers with commit privileges that have directly contributed to the project in one way or another.

Image	Name	Email
	Salah Ahmed	salah.ahmed@domain.com

Contributors

There are no contributors listed for this project. Please check back again later.

Copyright © 2025SE411-Tech. .

Example plugin: Create an executable

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-shade-plugin</artifactId>
            <version>3.5.1</version>
            <executions>
                <execution>
                    <phase>package</phase>
                    <goals>
                    <goal>shade</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

This plugin (using the shade goal) will create a “fat” executable (at the **package** phase) that contains all the required libraries embedded

```
\2024 Spring\SE411> java -classpath .\maven-03-dependencies-0.0.1-SNAPSHOT.jar psu.edu.se411.App
```

Source Code: maven-03-dependencies

Exception handling: Motivations

When a program runs into a runtime error, the program terminates abnormally.

How can you handle the runtime error so that the program can continue to run or terminate gracefully?

-> Exception handling

Exception-Handling Overview

Example: Here we get a runtime error

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    System.out.print("Enter two integers: ");

    int number1 = input.nextInt();
    int number2 = input.nextInt();

    // if number2 is 0, we get a runtime error
    System.out.println(number1 + " / " + number2 + " is " + (number1 / number2));
}
```

Fix it using an if statement

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in); // Prompt the user to enter two integers
    System.out.print("Enter two integers: ");
    int number1 = input.nextInt();
    int number2 = input.nextInt();
    if (number2 != 0)
        System.out.println(number1 + " / " + number2 + " is " + (number1 / number2));
    else
        System.out.println("Divisor cannot be zero"); }
}
```

Exception Advantages

In the previous example we can write if..then..else because we know what error can occur.

In other, more complex scenarios, we cannot know which error may happen, for example when we connect to a database, write into a file, read from some hardware, etc.

In such situations, **exceptions** are the solution.

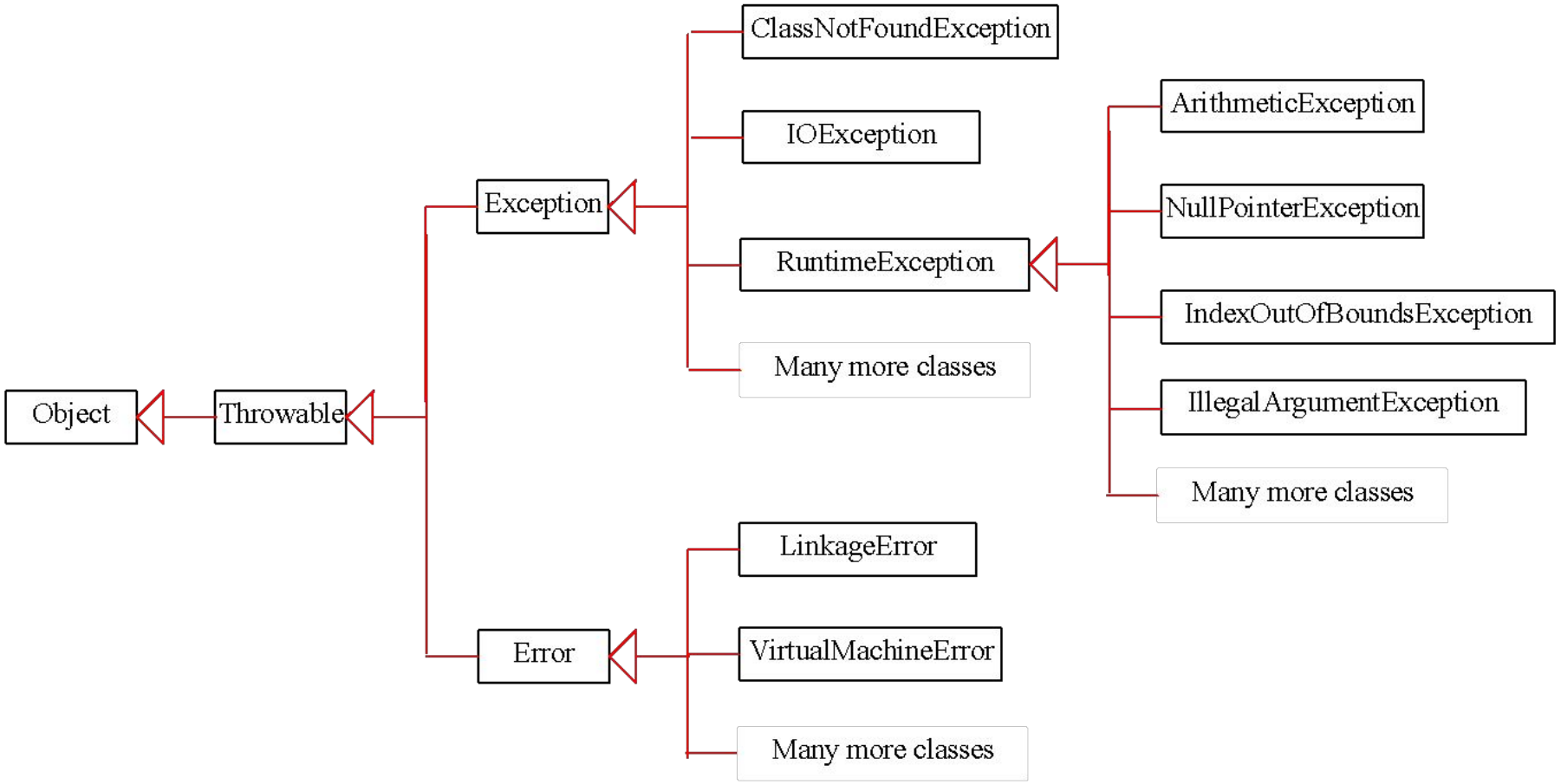
Example: Handling InputMismatchException

By handling **InputMismatchException**, your program will continuously read an input until it is correct.

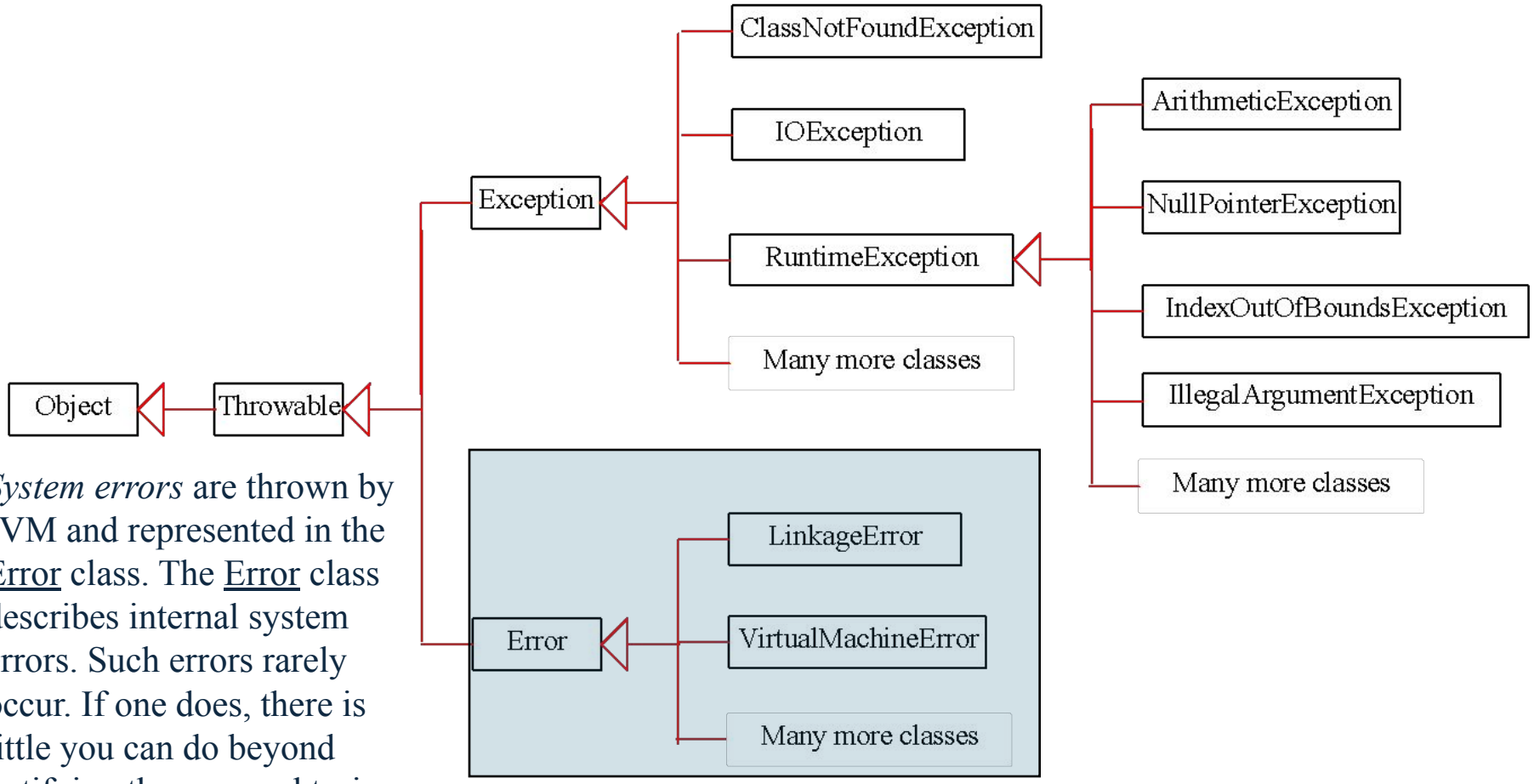
```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    boolean continueInput = true;
    do {
        try {
            System.out.print("Enter an integer: ");
            int number = input.nextInt();
            // Display the result
            System.out.println( "The number entered is " + number);
            continueInput = false;
        } catch (InputMismatchException ex) {
            System.out.println("Try again. (" + "Incorrect input: an integer is required)");
            input.nextLine(); // discard input
        }
    } while (continueInput);
}
```

Note: the developer needs to know which exception class to use.

Exception Types



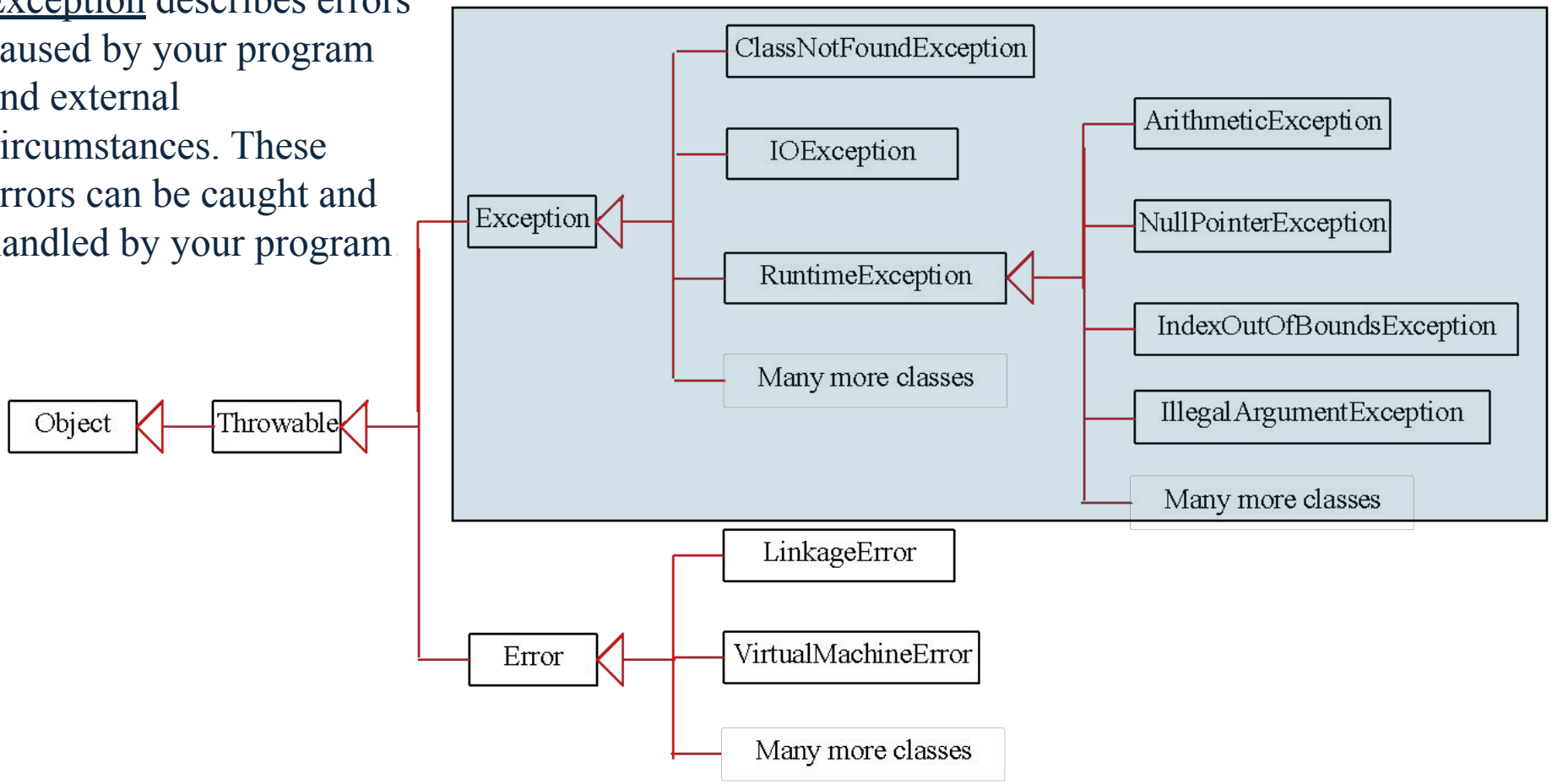
System Errors



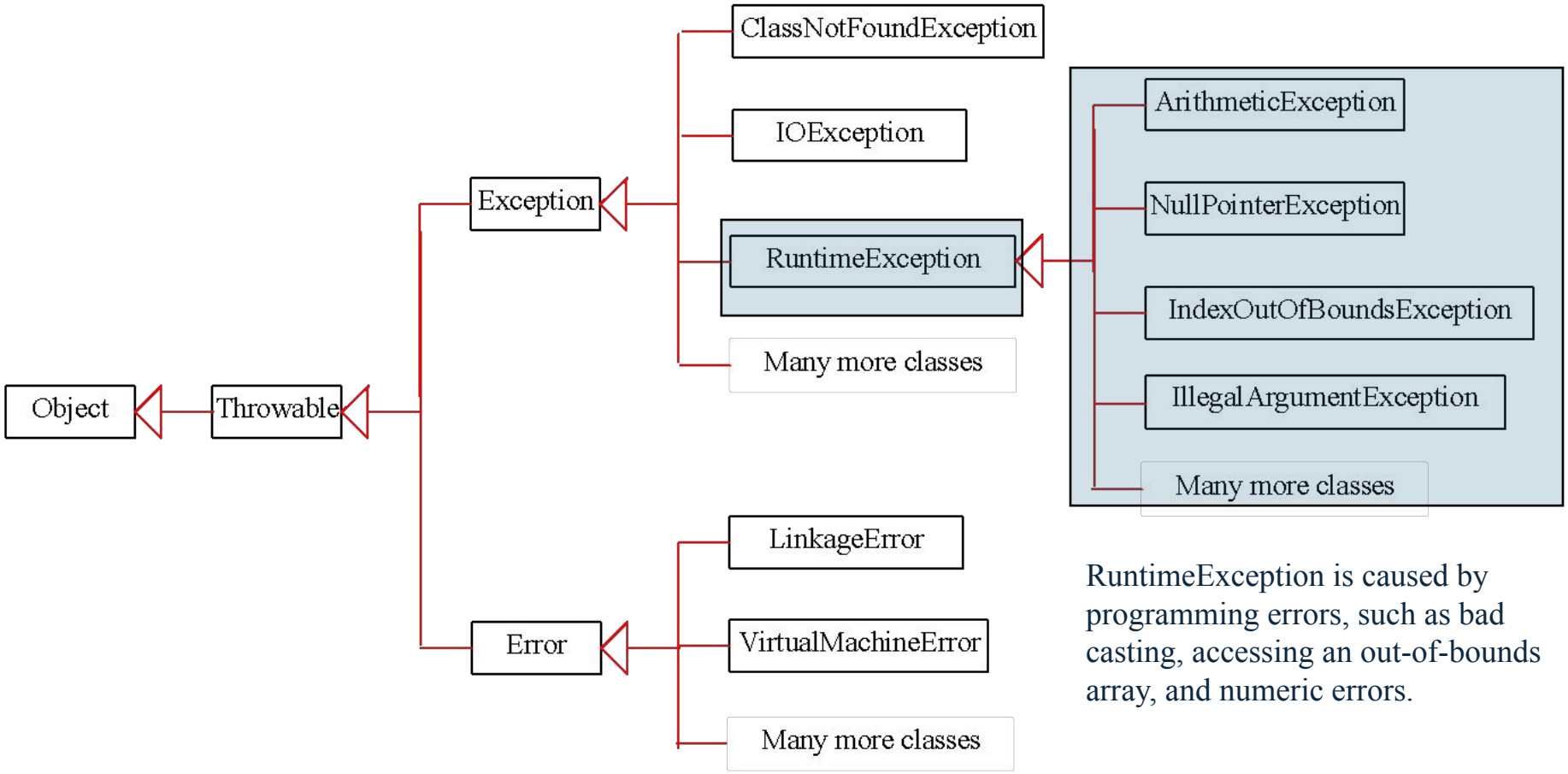
System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

Exceptions

Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program



Runtime Exceptions



RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

Checked Exceptions vs. Unchecked Exceptions

RuntimeException, Error and their subclasses are known as *unchecked exceptions*. All other exceptions are known as *checked exceptions*, meaning that the compiler forces the programmer to check and deal with the exceptions.

Unchecked Exceptions

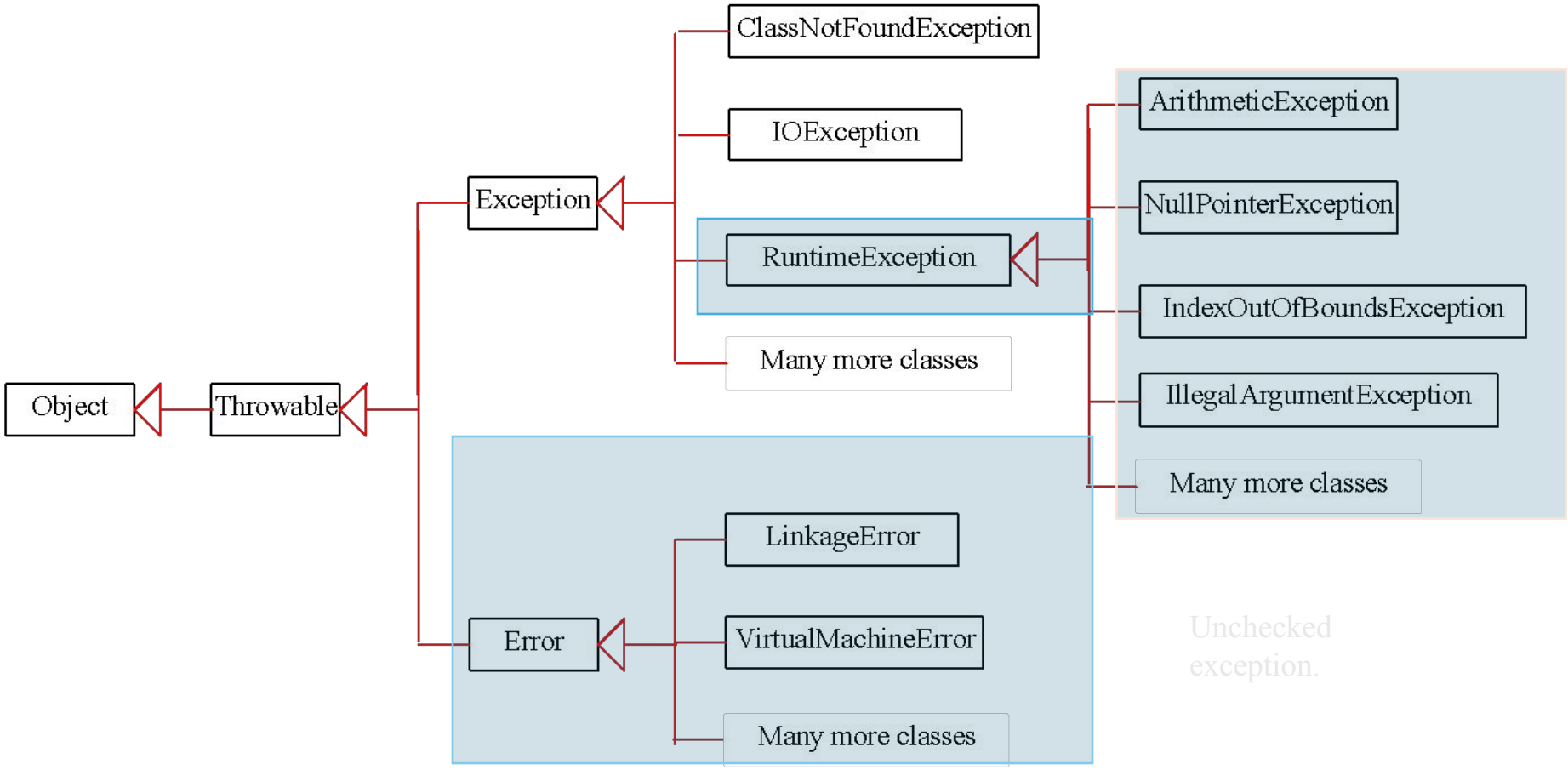
In most cases, unchecked exceptions reflect programming logic errors that are not recoverable. For example:

- a **NullPointerException** is thrown if you access a method through a reference variable before an object is assigned to it;
- an **IndexOutOfBoundsException** is thrown if you access an element in an array outside the bounds of the array.

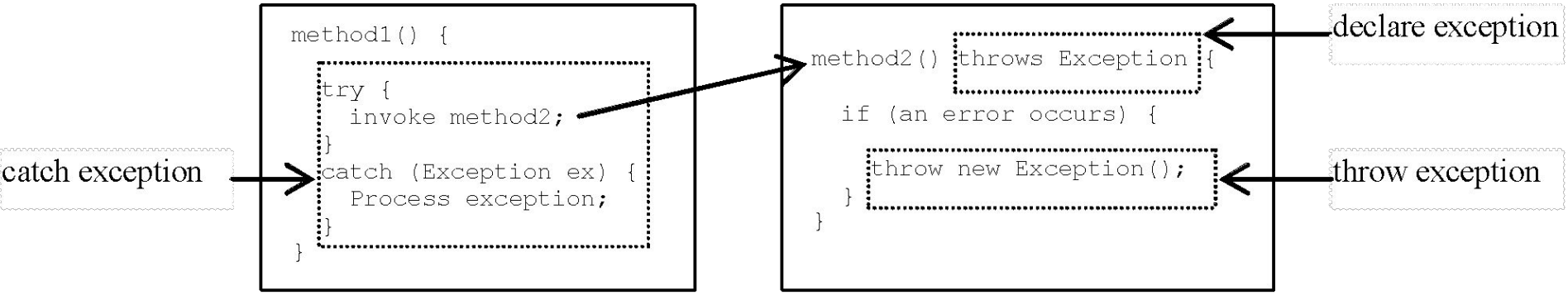
These are the **logic errors** that should be corrected in the program.

Unchecked exceptions can occur anywhere in the program. To avoid cumbersome overuse of try-catch blocks, **Java does not mandate you to write code to catch unchecked exceptions**.

Unchecked Exceptions



Declaring, Throwing, and Catching Exceptions



Declaring Exceptions

Every method must state the types of **checked** exceptions it might throw. This is known as *declaring exceptions*.

```
public void myMethod()  throws IOException

public void myMethod() throws IOException,
                              DBException
```

Throwing Exceptions

When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as *throwing an exception*. Here is an example,

```
throw new SomeException();
```

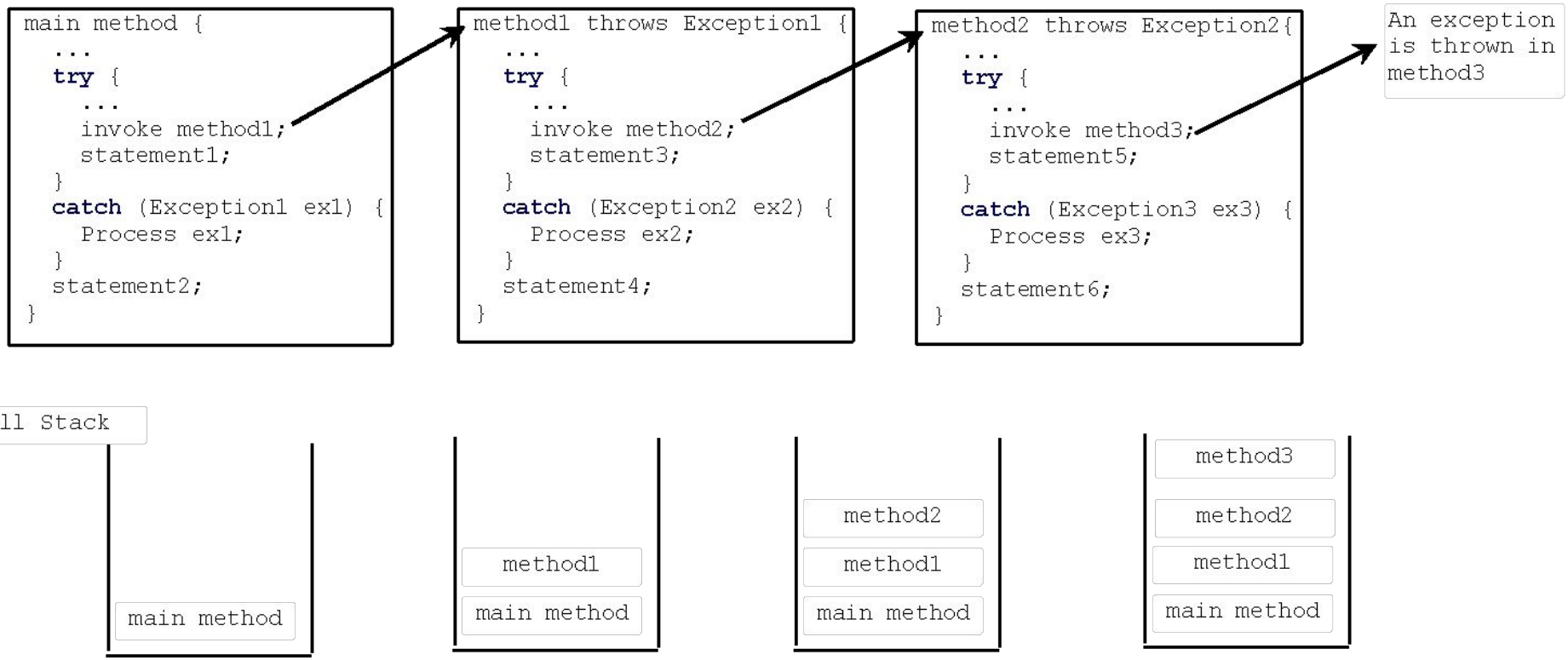
or

```
SomeException ex = new SomeException();
throw ex;
```

Catching Exceptions

```
try {
  statements;  // Statements that may throw exceptions
}
catch (Exception1 exVar1) {
  handler for exception1;
}
catch (Exception2 exVar2) {
  handler for exception2;
}
...
catch (ExceptionN exVar3) {
  handler for exceptionN;
}
```

Catching Exceptions



Catch or Declare Checked Exceptions

Suppose p2 is defined as follows:

```
void p2() throws IOException {
  if (a file does not exist) {
     throw new IOException("File does not exist");
  }

  ...
}
```

Catch or Declare Checked Exceptions

Java forces you to deal with checked exceptions. If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method. For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1(){
  try {
    p2();
  }
  catch (IOException ex) {
    ….
  }
}
```

(a)

or

```
void p1() throws IOException {

  p2();

}
```

(b)

Example: Declaring, Throwing, and Catching Exceptions

- Objective: This example demonstrates declaring, throwing, and catching exceptions by modifying the setRadius method. setRadius throws an exception if radius is negative.

```
public class CircleWithException {
  /** The radius of the circle */
  private double radius;


  /** Construct a circle with a specified radius */
  public CircleWithException(double newRadius) {
    setRadius(newRadius);
}
```

```
  /** Set a new radius */
  public void setRadius(double newRadius)
      throws IllegalArgumentException {
    if (newRadius >= 0)
      radius = newRadius;
    else
      throw new IllegalArgumentException(
        "Radius cannot be negative");
  }

}
```

```
public class CircleWithException {
   /** The radius of the circle */
   private double radius;


   /** Construct a circle with a specified radius */
   public CircleWithException(double newRadius) {
     setRadius(newRadius);
}
```

V

s

```
  /** Set a new radius */
  public void setRadius(double newRadius)
        throws IllegalArgumentException {
      if (newRadius >= 0)
         radius = newRadius;
      else
         throw new IllegalArgumentException(
              "Radius cannot be negative");
   }

}
```

```
public class CircleWithException {
   /** The radius of the circle */
   private double radius;


   /** Construct a circle with a specified radius */
   public CircleWithException(double newRadius) {
      if (newRadius >= 0)
        radius = newRadius;
     else
        throw new IllegalArgumentException(
             "Radius cannot be negative");
   }

   /** Set a new radius */
   public void setRadius(double newRadius)
         throws IllegalArgumentException {
       if (newRadius >= 0)
          radius = newRadius;
       else
          throw new IllegalArgumentException(
               "Radius cannot be negative");
    }

}
```

Rethrowing Exceptions

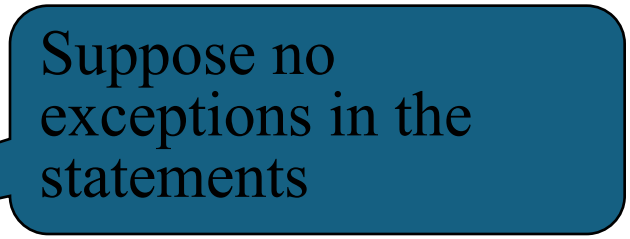
```
try {
   statements;
}
catch(SomeException ex) {
   perform operations before exits;
   throw ex;
}
```

The `finally` Clause

```
try {
   statements;
}
catch(SomeException ex) {
   handling ex;
}
finally {
   finalStatements;
}
```

animation

Trace a Program Execution




```
try {
  statements;
}
catch(SomeException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

animation

Trace a Program Execution

```
try {
  statements;
}
catch(SomeException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed

animation

Trace a Program Execution

```
try {
  statements;
}
catch(SomeException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Next statement in the method is executed

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Suppose an exception of type Exception1 is thrown in statement2

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The exception is handled.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The next statement in the method is now executed.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

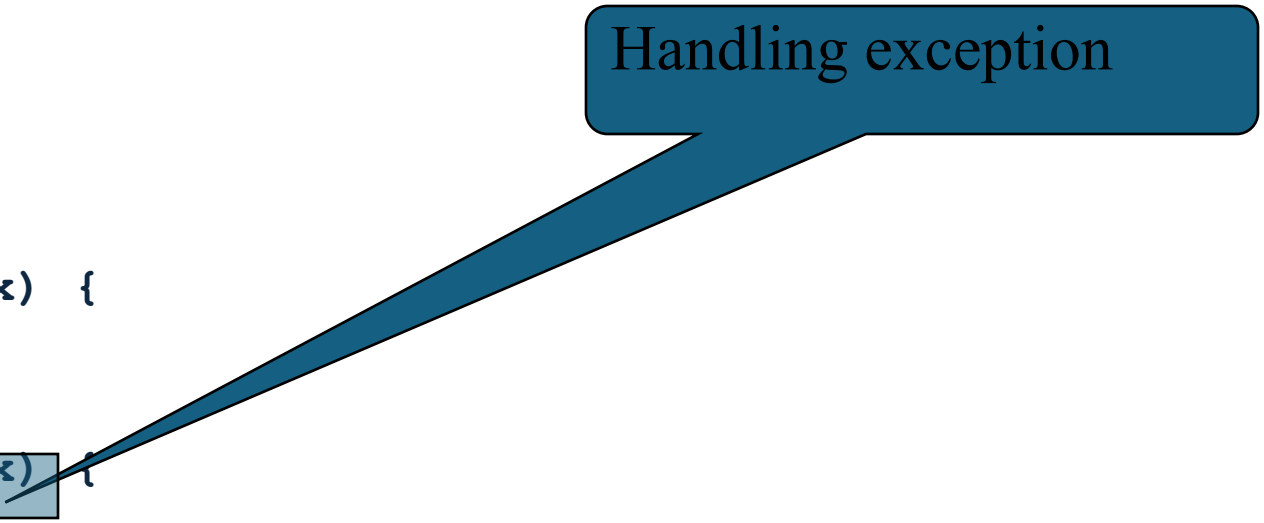
statement2 throws an exception of type Exception2.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

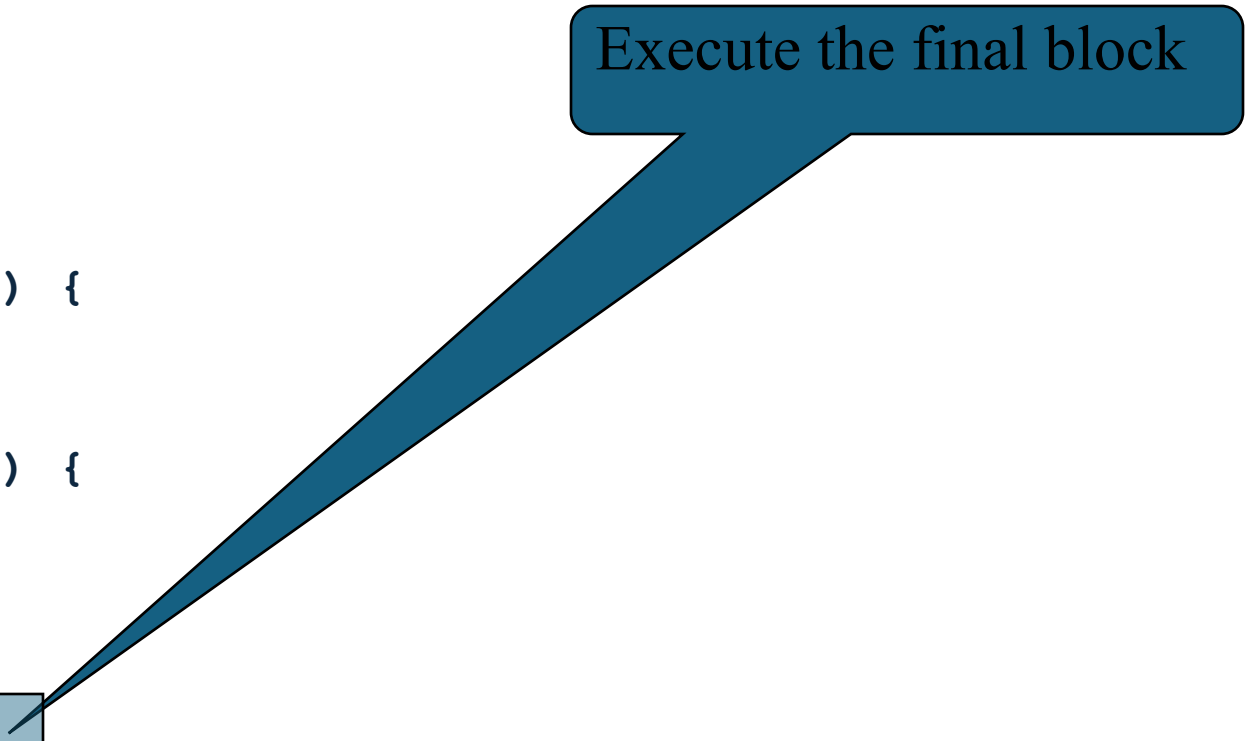


animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```



animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Rethrow the exception and control is transferred to the caller

Caution When Using Exceptions

- Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify.
- Be aware, however, that exception handling usually requires more time and resources because it requires instantiating a new exception object, rolling back the call stack, and propagating the errors to the calling methods.

When to Throw Exceptions

- An exception occurs in a method. If you want the exception to be processed by its caller, you should create an exception object and throw it. If you can handle the exception in the method where it occurs, there is no need to throw it.

When to Use Exceptions (anti-pattern)

When should you use the try-catch block in the code? You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {
  System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
  System.out.println("refVar is null");
}
```

When to Use Exceptions

is better to be replaced by

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```

Defining Custom Exception Classes

g
3

- Use the exception classes of the standard java API whenever possible.
- Define custom exception classes if the predefined classes are not sufficient.
- Define custom exception classes by extending Exception or a subclass of Exception.

Custom Exception Class Example

In the previous CircleWithException example, the setRadius method throws an exception if the radius is negative. Suppose you wish to pass the radius to the handler (caller), you have to create a custom exception class.

```
public class InvalidRadiusException extends Exception {
    private double radius;

    /** Construct an exception */
    public InvalidRadiusException(double radius) {
        super("Invalid radius " + radius);
        this.radius = radius;
     }

    /** Return the radius */
    public double getRadius() {
        return radius;
    }
}
```

Custom Exception Class Example (2)

```
public class Circle {
  /** The radius of the circle */
  private double radius;

  /** The number of the objects created
*/
  private static int numberOfObjects = 0;

  /** Construct a circle with radius 1 */
  public Circle () {
    this(1.0);
  }

  /** Construct a circle with a radius */
  public Circle (double newRadius) {
    try {
      setRadius(newRadius);
      numberOfObjects++;
    }
    catch (InvalidRadiusException ex) {
      ex.printStackTrace();
    }
  }

  /** Return radius */
  public double getRadius() {
    return radius;
  }

  /** Set a new radius */
  public void setRadius(double newRadius)
      throws InvalidRadiusException {
    if (newRadius >= 0)
      radius =  newRadius;
    else
      throw new
InvalidRadiusException(newRadius);
  }

  /** Return numberOfObjects */
  public static int getNumberOfObjects() {
    return numberOfObjects;
  }

  /** Return the area of this circle */
  public double findArea() {
    return radius * radius * 3.14159;
  }
}
```

Assertions

g
1

An assertion (تأكد من فرضية) is a Java statement that enables you to assert an assumption about your program.

An assertion contains a **Boolean** expression **that should be true during program execution**.

Assertions can be used to assure program correctness and avoid logic errors during development as a sort of testing at development time.

Declaring Assertions

An *assertion* is declared using the Java keyword assert (since JDK 1.4) as follows:

assert *assertion*; or
assert *assertion* : *detailMessage*;

where ***assertion*** is a Boolean expression and ***detailMessage*** is a primitive-type or an Object value.

Executing Assertions

When an assertion statement is executed, Java evaluates the assertion. If it is false, an **AssertionError** will be thrown. The AssertionError class has a no-arg constructor and seven overloaded single-argument constructors of type int, long, float, double, boolean, char, and Object.

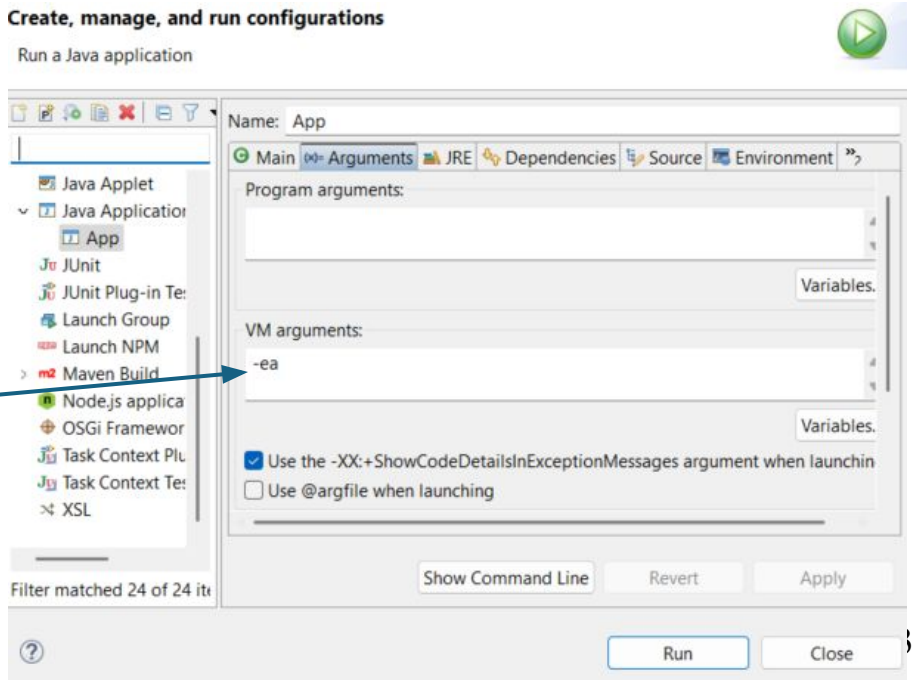
```
Exception in thread "abc" java.lang.AssertionError: Some Message
```

For the first assert statement with no detail message, the no-arg constructor of AssertionError is used. For the second assert statement with a detail message, an appropriate AssertionError constructor is used to match the data type of the message. Since AssertionError is a subclass of Error, when an assertion becomes false, the program displays a message on the console and exits.

Executing Assertions Example

```
public class AssertionDemo {
  public static void main(String[] args) {
    int i; int sum = 0;
    for (i = 0; i < 10; i++) {
      sum += i;
    }
    assert i == 10;
    assert sum > 10 && sum < 5 * 10 : "sum is " + sum;
  }
}
```

You need to activate assertions by the JVM by adding –ea in the VM arguments



Running Programs with Assertions

By default, the assertions are disabled at runtime. To enable it, use the switch –enableassertions, or –ea for short, as follows:

java –ea AssertionDemo

Assertions can be selectively enabled or disabled at class level or package level. The disable switch is –disableassertions or –da for short. For example, the following command enables assertions in package package1 and disables assertions in class Class1.

java –ea:package1 –da:Class1 AssertionDemo

Using Exception Handling or Assertions

Assertion should not be used to replace exception handling.

Exception handling deals with unusual circumstances during **program execution**.

Assertions are to assure the correctness of the **program logic**.

Exception handling addresses robustness and assertion addresses correctness.

Like exception handling, assertions are not used for normal tests, but for internal consistency and validity checks. Assertions are checked at runtime and can be turned on or off at startup time.

Using Exception Handling or Assertions, cont.

Do not use assertions for argument checking in public methods. Valid arguments that may be passed to a public method are considered to be part of the method's contract. The contract must always be obeyed whether assertions are enabled or disabled. For example, the following code in the Circle class should be rewritten using exception handling.

```
public void setRadius(double newRadius) {
  assert newRadius >= 0;
  radius =  newRadius;
}
```

Using Exception Handling or Assertions, cont.

Use assertions to reaffirm assumptions. This gives you more confidence to assure correctness of the program.

A common use of assertions is to replace assumptions with assertions in the code.

Using Exception Handling or Assertions, cont.

Another good use of assertions is to place assertions in a switch statement without a default case. For example,

```
switch (month) {
  case 1: ... ; break;
  case 2: ... ; break;
  ...
  case 12: ... ; break;
  default: assert false : "Invalid month: " + month
}
```

Defensive Programming

- Assume the inputs to your code may be wrong and protect against them.
- Validate data at the boundaries of your module (a 'barricade').
- Inside the barricade, data can be trusted; outside it cannot.
- **Techniques:**
  - Check parameters and return values.
  - Handle bad input explicitly: reject, correct, or log it.
  - Fail fast: surface errors early, near their cause.
- **Goal: code that degrades safely, not mysteriously.**

Design by Contract

- Think of every method as a contract between the caller and the method.
- Preconditions: what the caller must guarantee before calling.
- Postconditions: what the method guarantees on return.
- Invariants: what stays true for an object throughout its life.
- **Assertions are how you encode and check these contracts in code.**
- **Clear contracts turn 'it broke somewhere' into 'the caller violated the precondition'.**

Using AI for quality code review

- **Prompt: “I want you to analyze the entire codebase of this project with a focus on error handling. Please perform a deep review and evaluate whether:**

1. Error handling follows industry best practices

- **Proper use of exceptions or error-return patterns**
- **Clear separation between recoverable and non-recoverable errors**
- **Meaningful error messages**
- **Avoidance of silent failures**
- **Proper logging strategy**
- **No overuse or misuse of try/catch blocks**

2. A unified and consistent error-handling approach is applied across all modules

- **Consistent patterns, naming conventions, and structures**
- **Centralized error-handling mechanisms (if applicable)**
- **Consistent logging format and severity levels**
- **Consistent use of custom error types or error codes**

Using AI for quality code review (2)

3. Identify any deviations, anti-patterns, or risks

- **Highlight files or components where error handling is missing, or poorly implemented**
- **Point out duplicated logic or conflicting patterns**
- **Identify areas where error handling could cause maintainability or reliability issues**

4. Provide actionable recommendations

- **Suggest improvements or refactoring steps**
- **Propose a unified error-handling strategy for the entire project**
- **Provide examples or templates the team can adopt**

5. Deliver your findings in a structured report with:

- **A high-level summary**
- **Detailed findings per module/file**
- **A list of recommended improvements**
- **Optional code snippets demonstrating best-practice patterns**

Begin your analysis once you have fully parsed the codebase.

Result on DeepTutor code base

https://github.com/HKUDS/DeepTutor.git

Summary of Findings

Overall Score: 5.5/10 (Needs Improvement)

Category	Score
Exception Handling	5/10
Logging Strategy	6/10
API Error Responses	7/10
Consistency	4/10

Issue Counts

Severity	Count
CRITICAL	15
MAJOR	24
MINOR	31
Total	70

Key Findings

Key Findings

Critical Issues:

- **20 bare `except:` clauses** catching all exceptions silently
- **15 silent failure points** where errors are swallowed without logging
- **QueueFull handling** drops logs silently under load
- **Zero exception chaining** (`raise ... from e`) in entire codebase
- **Unhandled async exceptions** in document processing

Pattern Issues:

- **80+ print() statements** for errors instead of the logging system
- **Research module**: Only 11% logger usage (8 logger calls vs 62 print calls)
- **Limited HTTP status codes**: Only 400, 404, 500 used (no 422 for validation)

Most Affected Files

File	Critical Issues
src/api/routers/solve.py	4 silent swallows
src/api/routers/question.py	3 bare excepts
src/core/logging/handlers.py	QueueFull silent drop
src/tools/paper_search_tool.py	No API error handling
src/agents/research/agents/research_agent.py	62 print() calls

The full report includes detailed file-by-file findings, an anti-patterns catalog, an

Debugging as a Discipline

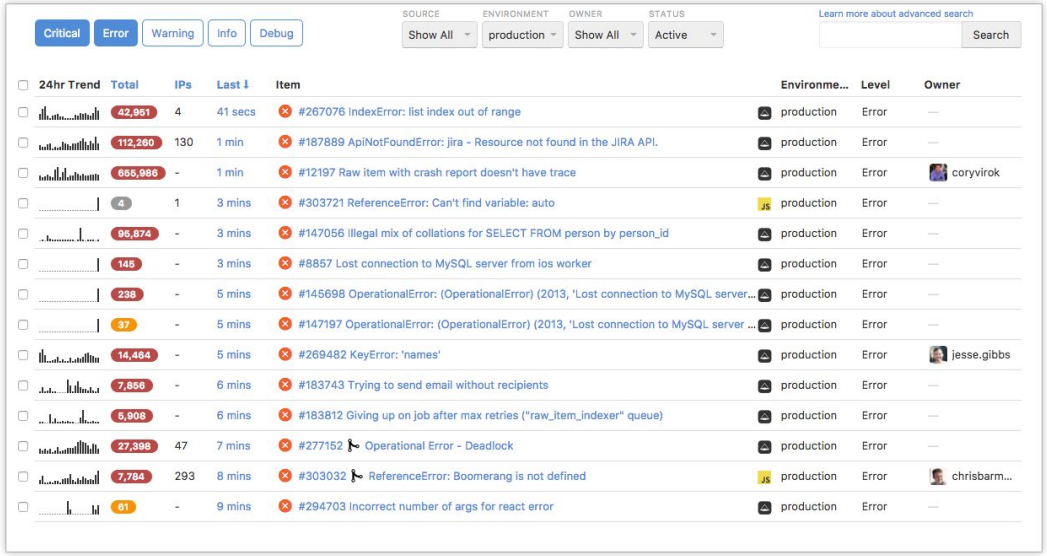
- Debugging is science, not guesswork.
- **The loop:**
  - Reproduce the bug reliably.
  - Isolate it (find the exact cause): narrow down where it happens.
  - Form a hypothesis about the cause.
  - Fix, then verify the fix and check for regressions.
- A bug is an opportunity: it reveals a gap in your understanding or your tests.
- **Use tools: debuggers, logging, and a failing unit test that pins the bug.**

Intro to logging

- **Definition of Logging**:
  The process of recording information about program execution, errors, and other significant events during runtime.

- **Importance**:
  It helps developers debug and monitor applications.
  Provides insights into program flow and state.
  Essential for troubleshooting and improving software reliability.

- **Types of Logging**:
  Informational (e.g., system activity)
  Debugging (e.g., variable states)
  Error (e.g., exceptions)



Key Logging Concepts in Java

Log Levels:

- **SEVERE**: Critical issues (e.g., application crash).
- **WARNING**: Potential problems (e.g., deprecated methods).
- **INFO**: General information (e.g., application startup).
- **DEBUG**: Detailed information for debugging.
- **TRACE**: Very detailed information (used for tracing fine-grained flow).

Loggers:

- A Logger is the library for logging messages.

Example:

```
import java.util.logging.*;

public class LoggingExample {
    private static final Logger logger =

Logger.getLogger(LoggingExample.class.getName());

    public static void main(String[] args) {
        logger.info(“Application started….");
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            logger.severe("Error occurred: " + e.getMessage());
        }
    }
}
```

Logging best practices

1. Log Level Usage:

•Use the appropriate level for each message (e.g., errors at SEVERE, routine information at INFO).

2. Never Log Sensitive Data:

•Be cautious of logging passwords, private keys, or other sensitive user data.

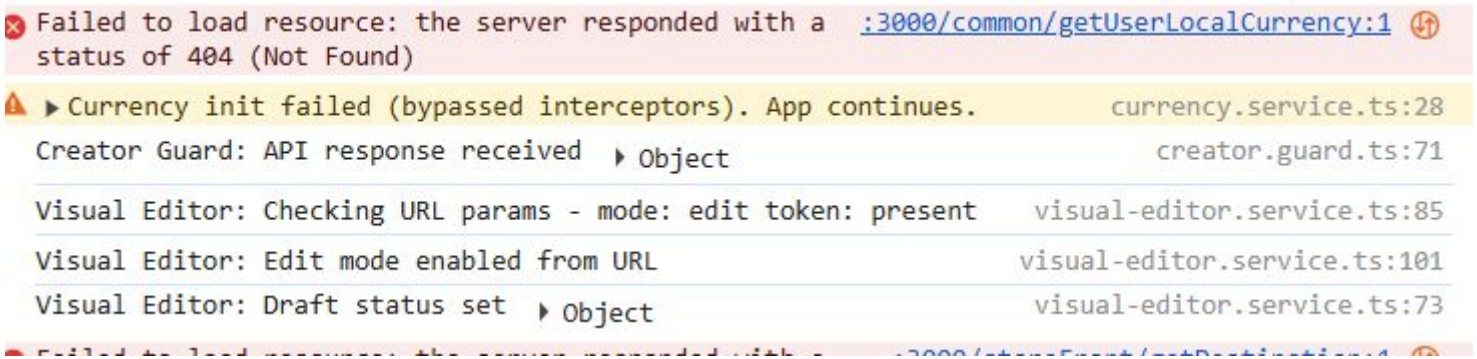
3. Use Structured Logs:

•Consider structured logs (e.g., JSON format) for easier parsing and analysis.

4. Optimize Performance:

•Limit excessive logging in performance-critical sections to avoid overhead.

•Use asynchronous logging where appropriate.



Logging sensitive data to the browser console is more critical

Verify using AI: Silent Failures

Spot the two anti-patterns:

```
try {
  int r = process(input);
  log.info("key=" + apiKey);
} catch (Exception e) {
  // ignore and continue
}
```



```
// Anti-pattern 1: swallowed
//   exception -> failures
//   vanish silently
// Anti-pattern 2: secret in
//   the log -> apiKey leaked

// Fix: catch specific types,
//   handle or rethrow, and
//   never log secrets / PII
```

AI loves broad catch blocks and chatty logs. Verify that exceptions are not swallowed and that logs never contain secrets or PII (Personal Identifiable Information).

Using AI to implement a full logging strategy

Prompt: “I want you to analyze the entire codebase of this project with a focus on logging. Your goal is to help me design and implement a complete, consistent, and high-quality logging strategy. Please perform a deep review and evaluate the following:

1. Current Logging Practices
   - Identify where logging is implemented and where it is missing
   - Detect inconsistencies in log formats, levels, structure, or message style
   - Highlight any misuse of logging levels (e.g., info vs debug vs error)
   - Identify redundant, noisy, or low-value logs
   - Detect logs that expose sensitive information
2. Logging for Maintainability
   - Evaluate whether logs help with debugging, tracing, and root-cause analysis
   - Check if logs include enough context (e.g., identifiers, state, error details)
   - Identify gaps where logs should be added to improve observability
   - Assess whether logs follow a predictable structure that supports automated analysis
3. Logging for Customer Behavior and Product Insights
   - Identify opportunities to log user actions, flows, and feature usage
   - Suggest what behavioral events should be captured (e.g., clicks, errors, drop-offs)
   - Ensure logs can support analytics, dashboards, and product-usage metrics
   - Recommend how to separate operational logs from behavioral/analytics logs

Using AI to implement a full logging strategy (2)

4. Best-Practice Alignment
   - Compare the current implementation with industry best practices for the project's language and architecture
   - Recommend standardized log formats (JSON, structured logs, correlation IDs, etc.)
   - Suggest a unified logging framework or library if needed
   - Propose naming conventions, metadata standards, and log-level guidelines
5. Actionable Improvements
   - Provide a unified logging strategy for the entire project
   - Suggest refactoring steps to make logging consistent across modules
   - Provide example code snippets showing the recommended logging pattern
   - Highlight high-priority fixes and long-term improvements
6. Deliverables

   Please provide your findings in a structured report with:
   - A high-level summary
   - Detailed findings per module/file
   - A proposed unified logging strategy
   - Code examples demonstrating the recommended approach
   - A prioritized implementation plan

Begin your analysis once you have fully parsed the codebase.

Should I proceed with implementing the logging improvements, or would you like me to adjust the analysis/plan first? : "write the plan into a new file "logging improvement.md" then wait for instruction"

Result on DeepTutor code base

https://github.com/HKUDS/DeepTutor.git

DeepTutor Logging Analysis Report

Executive Summary

DeepTutor is a full-stack AI tutoring platform with a **Python/FastAPI backend** and **Next.js frontend**. The codebase has a **well-designed custom logging infrastructure** in `src/core/logging/`, but **inconsistent adoption** across modules, with **67 identified logging gaps** ranging from silent exception handlers to missing audit trails.

Key Findings

- **Strengths**: Custom unified logger with WebSocket streaming, color-coded output, LLM token tracking, structured JSON logging
- **Weaknesses**: 12 unlogged error handlers, 15 API endpoints without request logging, 8 silent `except: pass` patterns
- **Critical Gaps**: No frontend logging, missing audit trails for config changes, no behavioral analytics

1. Current Logging Practices Analysis

1.1 Logging Infrastructure (Well-Implemented)

Component	Location	Purpose
`Logger` class	`src/core/logging/logger.py`	Unified format `[Module] Symbol Message`
`WebSocketLogHandler`	`src/core/logging/handlers.py`	Real-time log streaming to frontend
`LLMStats`	`src/core/logging/llm_stats.py`	Token usage & cost tracking
`LightRAGLogForwarder`	`src/core/logging/lightrag_forward.py`	RAG library log integration

1.2 Log Levels & Symbols

```
DEBUG   (·)  - Detailed diagnostic info
INFO    (○)  - General operational events
SUCCESS (✓)  - Successful completion
WARNING (⚠)  - Potential issues
ERROR   (✗)  - Failures requiring attention
CRITICAL(✗)  - System-breaking failures
PROGRESS(→)  - Long-running operation status
```

1.3 Files WITH Proper Logging (37 files)

- `src/agents/solve/*` - Good logging in agent lifecycle
- `src/agents/research/research_pipeline.py` - Excellent stage progress logging
- `src/api/routers/solve.py` - Good WebSocket log streaming
- `src/core/logging/*` - Complete infrastructure

1.4 Files MISSING Logging (Critical)

File	Issue	Severity
`src/api/routers/dashboard.py`	No logging module imported	High
`src/api/routers/settings.py`	No audit logging for config changes	Critical
`src/api/routers/notebook.py`	No CRUD operation logging	High

2. Detailed Findings by Module

2.1 API Layer (`src/api/`)

Silent Exception Handlers (CRITICAL)

```
# src/api/routers/question.py:245
except: pass  # WebSocket error silently swallowed

# src/api/routers/ideagen.py:399
except (RuntimeError, WebSocketDisconnect, ConnectionError): pass

# src/api/routers/solve.py:90-91
except: pass  # Queue operations fail silently
```

Missing Endpoint Logging

- `src/api/routers/notebook.py`: All 8 endpoints lack request/response logging
- `src/api/routers/dashboard.py`: No logger instance at all

4.2 Replacing Silent Exception Handlers

Before (`src/api/routers/question.py:245`):

```
except:
    pass
```

After:

```
except Exception as e:
    logger.warning(f"WebSocket cleanup error (non-critical): {e}")
```

5. Prioritized Implementation Plan

Phase 1: Critical Fixes (High Priority)

Task	Files	Impact
1. Replace all `except: pass` with logged handlers	`question.py`, `ideagen.py`, `solve.py`	Prevents silent failures
2. Add logger to `dashboard.py`	`src/api/routers/dashboard.py`	Basic observability
3. Add audit logging to settings endpoints	`src/api/routers/settings.py`	Security compliance
4. Log all file I/O errors	`history.py`, `notebook_manager.py`	Debugging

Summary

The DeepTutor codebase has excellent logging **infrastructure** but **inconsistent adoption**. The custom logging system in `src/core/logging/` is well-designed with WebSocket streaming, structured output, and LLM cost tracking. However, 67 gaps exist across the codebase, with critical issues including:

1. **8 silent exception handlers** that swallow errors
2. **No logging in dashboard.py**
3. **No audit trails** for configuration changes
4. **No frontend logging** infrastructure

The recommended approach is a 3-phase implementation starting with critical fixes (silent exceptions, missing loggers) and progressing to full coverage and analytics capabilities.